بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

18 – Characters and Strings

Introduction

- Computers were invented to crunch numbers
- But as soon as they became commercially viable they were used to process text
- Most computers today offer 8-bit bytes to represent characters
  - American Standard Code for Information Interchange (ASCII)

ASCII Representation of Characters

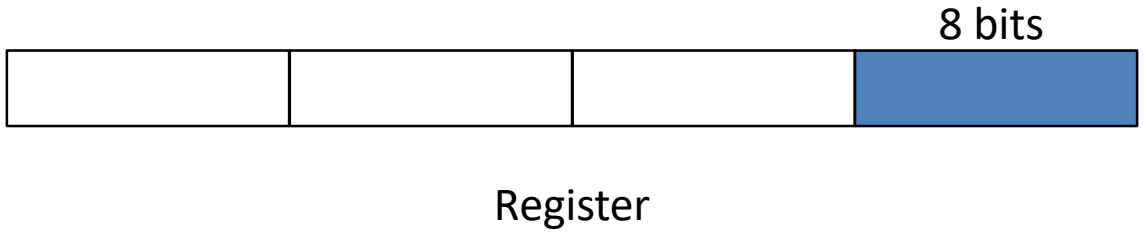
ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter	ASCII value	Char-acter
32	space	48	0	64	@	80	P	96	`	112	p
33	!	49	1	65	A	81	Q	97	a	113	q
34	"	50	2	66	B	82	R	98	b	114	r
35	#	51	3	67	C	83	S	99	c	115	s
36	$	52	4	68	D	84	T	100	d	116	t
37	%	53	5	69	E	85	U	101	e	117	u
38	&	54	6	70	F	86	V	102	f	118	v
39	'	55	7	71	G	87	W	103	g	119	w
40	(	56	8	72	H	88	X	104	h	120	x
41	)	57	9	73	I	89	Y	105	i	121	y
42	*	58	:	74	J	90	Z	106	j	122	z
43	+	59	;	75	K	91	[	107	k	123	{
44	,	60	<	76	L	92	\	108	l	124	|
45	-	61	=	77	M	93	]	109	m	125	}
46	.	62	>	78	N	94	^	110	n	126	~
47	/	63	?	79	O	95	_	111	o	127	DEL

ASCII vs Binary Numbers

- Represent numbers as strings of ASCII digits instead of as integers
- How much does storage increase if number 1 billion is represented in ASCII versus a 32-bit integer?
  - One billion (1,000,000,000), would take 10 ASCII digits, each 8 bits long
    - Storage would be (10 × 8)/32 or 2.5
- HW to add, subtract, multiply, and divide decimal numbers is difficult and would consume more energy
- Such difficulties explain why binary is natural and decimal is bizarre

Byte Extraction

- Because of popularity of text, MIPS provides instructions to move bytes
  - Load byte (lb) loads a byte from memory, placing it in the rightmost 8 bits of a register
  - Store byte (sb) takes a byte from rightmost 8 bits of a register and writes it to memory



Byte Extraction

- Copy a byte with the code sequence

```
lb $t0,0($s0) # Read byte from source
sb $t0,0($s1) # Write byte to destination
```

Strings

- Characters are normally combined into strings, which have a variable number of characters
- Three choices for representing a string
  1. First position is reserved to give length of string
  2. An accompanying variable has length of string
  3. Last position is indicated by character used to mark end of a string

Strings

- C uses the third choice, terminating a string with a byte whose value is 0 (named null in ASCII)
  - e.g., string “Cal” is represented in C by the following 4 bytes, shown as decimal numbers: 67, 97, 108, 0
- Java uses the first option

Example

- Procedure `strcpy` copies string y to string x using the null byte termination convention of C. Base addresses for arrays x and y in $a0 and $a1, i in $s0.

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while ((x[i] = y[i])!='\0')
    i += 1;
}
```

Example (cont.)

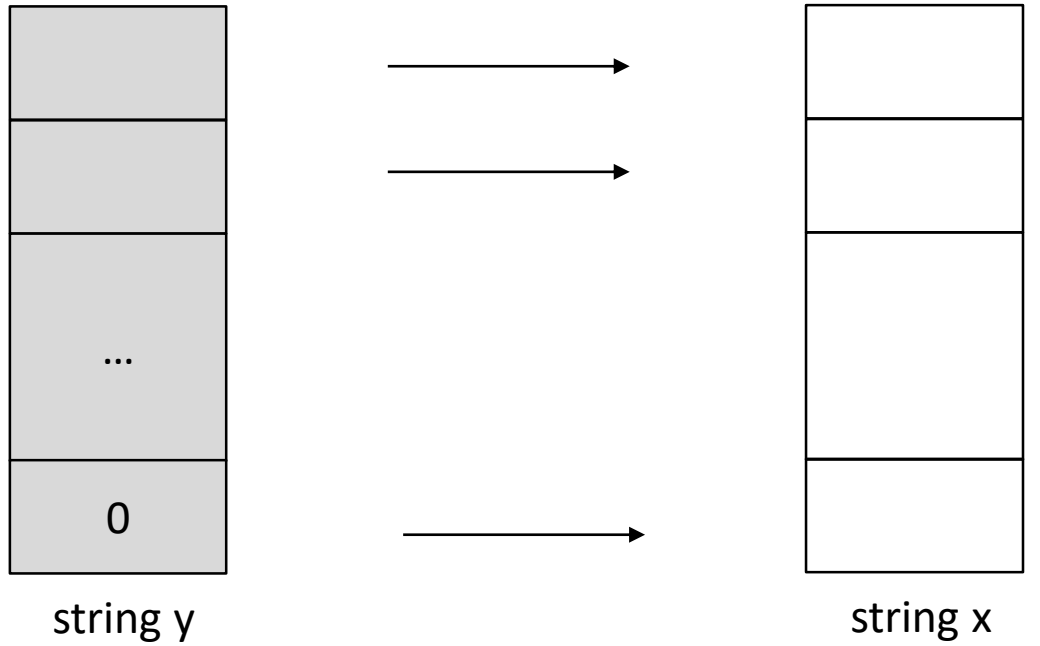
- Procedure `strcpy` copies string y to string x using the null byte termination convention of C. Base addresses for arrays x and y in $a0 and $a1, i in $s0.

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

Example (cont.)

- Copy character-by-character from y-string to x-string.

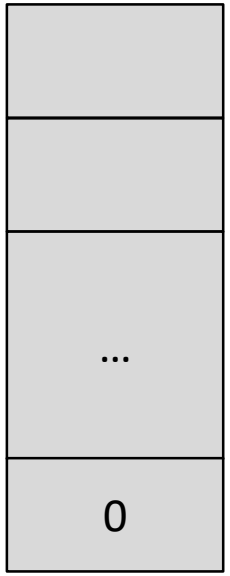
```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```



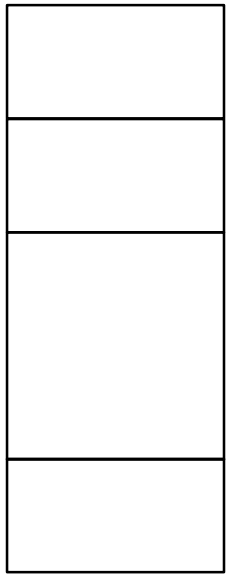
Example (cont.)

- Both x-string and y-string are in memory
  - Can not copy directly in memory

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```



string y

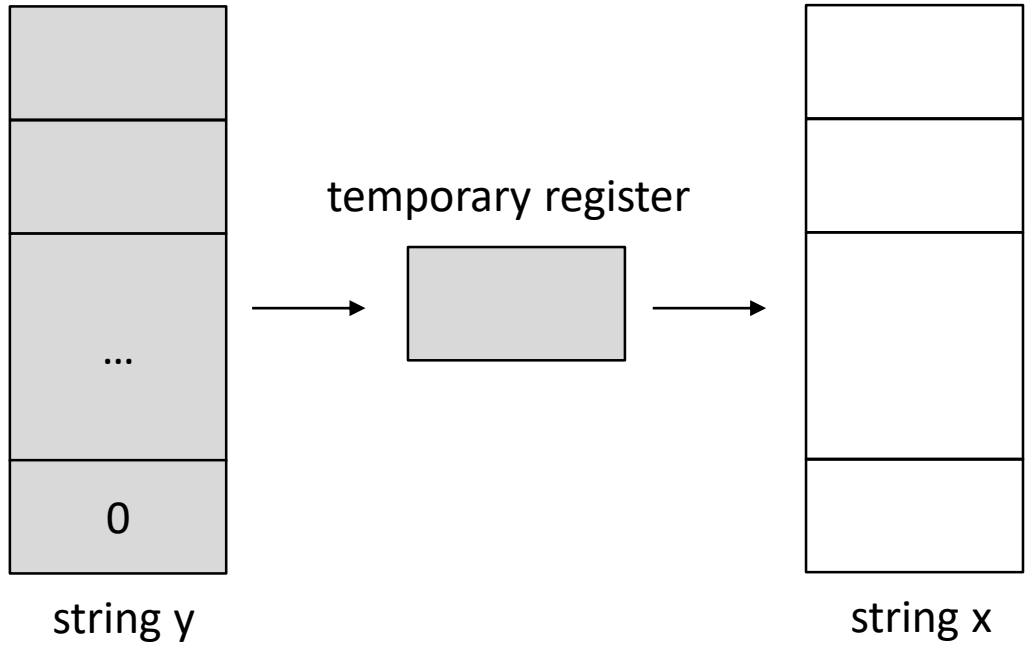


string x

Example (cont.)

- Use temporary register
  - Copy from y to register, then from register to x

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```



Example (cont.)

- Adjust SP and then saves register $s0 on stack

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
```

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

Example (cont.)

- Initialize i to 0,
  - adding 0 to 0 and placing sum in $s0

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
```

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

Example (cont.)

- Beginning of loop
- Address of y[i] is first formed by adding i to y[]

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
```

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

Example (cont.)

- Load character in y[i]
  - use lbu, to put character into $t2

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
    lbu  $t2, 0($t1)
```

Example (cont.)

- Address calculation of x[i] in $t3
- Character in $t2 is stored at that address

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
    lbu  $t2, 0($t1)
    add  $t3, $s0, $a0
    sb   $t2, 0($t3)
```

Example (cont.)

- Next, exit loop if character is 0
- i.e., we exit if it is last character of string

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
    lbu  $t2, 0($t1)
    add  $t3, $s0, $a0
    sb   $t2, 0($t3)
    beq  $t2, $zero, L2
```

Example (cont.)

- If not (character is not 0), increment i and loop back

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
    lbu  $t2, 0($t1)
    add  $t3, $s0, $a0
    sb   $t2, 0($t3)
    beq  $t2, $zero, L2
    addi $s0, $s0, 1
    j    L1
```

Example (cont.)

- If don’t loop back, it is last character of string
- Restore $s0 and SP

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
    lbu  $t2, 0($t1)
    add  $t3, $s0, $a0
    sb   $t2, 0($t3)
    beq  $t2, $zero, L2
    addi $s0, $s0, 1
    j    L1
L2: lw   $s0, 0($sp)
    addi $sp, $sp, 4
```

Example (cont.)

- Return

```
void strcpy (char x[], char y[])
{ int i;
  i = 0;
  while (y[i] !='\0'){
    x[i] = y[i];
    i += 1;
  }
}
```

```
strcpy:
    addi $sp, $sp, -4
    sw   $s0, 0($sp)
    add  $s0, $zero, $zero
L1: add  $t1, $s0, $a1
    lbu  $t2, 0($t1)
    add  $t3, $s0, $a0
    sb   $t2, 0($t3)
    beq  $t2, $zero, L2
    addi $s0, $s0, 1
    j    L1
L2: lw   $s0, 0($sp)
    addi $sp, $sp, 4
    jr   $ra
```

Characters and Strings in Java

- *Unicode* is a universal encoding of the alphabets of most human languages
- To be more inclusive, Java uses Unicode for characters
- By default, it uses 16 bits to represent a character

Load Halfword Instruction

- MIPS instruction set has explicit instructions to load and store 16-bit quantities, called *halfwords*
- *Load half* (lh)
  - loads a halfword from memory, placing it in rightmost 16 bits of a register
  - Treats the halfword as a signed number and thus sign-extends to fill the 16 left most bits of the register

Load Halfword Unsigned Instruction

- *Load halfword unsigned* (lhu)
  - Works with unsigned integers
  - more popular of the two

Store Halfword Unsigned Instruction

- *Store half* (sh)
  - Takes a halfword from the rightmost 16 bits of a register and writes it to memory
- Copy a halfword with the sequence

```
lhu $t0,0($sp) # Read halfword (16 bits) from source
sh $t0,0($gp)  # Write halfword (16 bits) to destination
```